

Exam Date & Time: 14-May-2025 (02:30 PM - 05:30 PM)



MANIPAL ACADEMY OF HIGHER EDUCATION

Manipal Institute of Technology, Bengaluru

BTech VI Semester

End Semester Examination - April / May -2025

PROGRAMMING IN MODERN C PLUS PLUS [NPT-3210]

Marks: 50

Duration: 180 mins.

PART-A

Answer all the questions.

- 1) Analyse the differences between C and C++ by comparing their structure, syntax, and features using a program to find size of int, float, char and double data type". (4)
 - A)
 - B) Apply the following conditions of managing the ages of students in a classroom that always contains precisely four individuals. write two distinct programs, one in C and its counterpart in C++, that perform the following actions:
 1. Define a fixed-size array capable of holding four integer values representing the ages of the students. (4)
 2. Populate this array with example age values for the four students.
 3. Display the contents of the array to the console, clearly labeling each age (e.g., "Student Age [0]: ...").
- C) Explain with program where the "Move constructor" of ResourceHolder would be automatically invoked by the compiler (without explicitly using std::move).. (2)
- 2) Create a BankAccount class in C++ that encapsulates the following:
 - a. Data: accountNumber (private, integer), balance (private, double)
 - b. Methods:
 - A)
 - deposit(amount): Adds amount to the balance (public).
 - withdraw(amount): Subtracts amount from the balance if sufficient funds are available, otherwise displays an error message (public).
 - getBalance(): Returns the current balance (public).
 - displayAccountInfo(): Displays the accountNumber and balance (public).(4)
 - B) You're reviewing a C++ codebase where multiple developers are working on different modules of a large object-oriented application. One developer has written member functions that frequently reference data members using this pointer explicitly, while another developer accesses data members directly without using this. You come across the following snippet used in both styles: (4)

```
this->m2 = k2;
```

```
m1 = k1;
```

Evaluate the implications of using the `this` pointer explicitly versus accessing member variables implicitly within member functions.

- C) Explain how access specifiers in a class support the principle of encapsulation in Object-Oriented Programming. (2)
- 3) You are designing a configuration for a multi-threaded application that needs to load settings from a file. This configuration should be accessed frequently from various parts of the application, and it's crucial that all parts of the application use the *same* configuration instance to maintain consistency. Evaluate the trade-offs between different implementation strategies for a Singleton class. (4)
- A)
- B) You are building a game engine, and you need a central point to manage and access the game's audio system (e.g., playing sound effects, controlling background music volume). Develop two distinct ways to provide access to this audio system from various parts of your game code: one approach using the Singleton pattern for the audio manager, and another approach *without* using the Singleton pattern. Compare the Singleton-based and non-Singleton-based access mechanisms with program in the context of a complex game engine. (4)
- C) Explain how the use of two static members-`nObjCons` (number of objects constructed) and `nObjDes` (number of objects destroyed)-helps in determining the number of live objects at any point in time. (2)
- 4) `class A {` (4)
- A) `public:`
`// ... members of A ...`
`};`
- `class B : public A {`
`public:`
`// ... members of B ...`
`};`
- `int main() {`
`A* pa = nullptr;`
`B* pb = nullptr;`
`void* pv = nullptr;`
- `pa = pb; // UPCAST: Okay`
- `// pb = pa; // DOWNCAST: error: '=': cannot convert from 'A *' to 'B *'`
- `pv = pa; // Okay, but lose the type for A * to void *`
`pv = pb; // Okay, but lose the type for B * to void *`
- `// pa = pv; // error: '=': cannot convert from 'void *' to 'A *'`
`// pb = pv; // error: '=': cannot convert from 'void *' to 'B *'`
- `return 0;`

```
}

```

Analyse the code snippet and explain why the upcast from a pointer to a derived class (B*) to a pointer to a base class (A*) is considered "Okay" by the compiler, while the direct downcast from a pointer to a base class (A*) to a pointer to a derived class (B*) results in a compiler error.

- B) Consider a drawing application with a base Shape class defining a fundamental draw() operation. You have implemented a Circle class that inherits from Shape and provides its own specific draw() implementation to render circles. Now, the requirement arises to introduce the ability to draw filled circles. Your colleague suggests adding a new method, draw(bool fill), exclusively to the Circle class, instead of altering the Shape class or its existing draw() method. (4)
Evaluate this proposal by writing code.
- C) Describe the fundamental difference in access control provided by the protected access specifier in a base class compared to private and public when considering derived classes? (2)
- 5) An organization needs to develop a salary processing application for its staff
• At present it has an engineering division only where Engineers and Managers work. Every Engineer reports to some Manager. Every Manager can also work like an Engineer
A) • The logic for processing salary for Engineers and Managers are different as they have different salary heads
• In future, it may add Directors to the team. Then every Manager will report to some Director. Every Director could also work like a Manager (4)
• The logic for processing salary for Directors will also be distinct
• Further, in future it may open other divisions, like Sales division, and expand the workforce
Evaluate the proposed design of using inheritance to model the organization's staff (Engineers inheriting from Manager, and potentially Managers inheriting from Director) for the salary processing application.
- B) Analyze the provided C++ code and its output, focusing on the behavior of the constructors, destructors, and the Print() virtual function. (4)
#include <iostream>
using namespace std;

class B {
int data_;
public:
B(int d) : data_(d) { cout << "B()" << endl; }
virtual ~B() { cout << "~B()" << endl; }
virtual void Print() { cout << data_; }
};

class D : public B {
int *ptr_;
public:
D(int d1, int d2) : B(d1), ptr_(new int(d2)) { cout << "D()" << endl; }
virtual ~D() { cout << "~D()" << endl; delete ptr_; }
virtual void Print() { B::Print(); cout << " " << *ptr_; }

```
};

int main() {
    B *p = new B(2);
    B *q = new D(3, 5);

    p->Print(); cout << endl;
    q->Print(); cout << endl;

    delete p;
    delete q;

    return 0;
}
```

- C) Describe which type of functions within a class is an entry typically created in the VFT. (2)

-----End-----